

Appendix

This appendix is available at the paper's website temporallogic.org/research/FPROGG/architecture.html.

The Formula Progression Function *prog* Alg. 1. Unnecessary recursive cases for the $\Box$ and $\Diamond$ operators were removed as FPROGG parses those operators into their logical duals. The algorithm as implemented runs in $\mathcal{O}(|\pi| \cdot 2^h)$ where $|\pi|$ is the length of the trace and h is the height of the formula's Abstract Syntax Tree (AST).

Algorithm 1 The Formula Progression Algorithm modified from [1]. In short, if φ does not contain any temporal operators, then $\pi \models \varphi$ iff $\pi[0] \models \varphi$. If φ does contain temporal modalities, then a recursive algorithm must be applied to ascertain if $\pi \models \varphi$ based on the structure of φ .

```

1: function PROG( $\pi, \varphi$ )
2:   if  $|\pi| > 1$  then return prog(prog( $\varphi, \pi[0]$ ),  $\pi[1 : end]$ )    ▷Recurse until
    $|\pi| = 1$ 
3:   else                                ▷if  $|\pi| = 1$  then recurse on the structure of  $\varphi$ 
4:     if  $\varphi = true$  then return true
5:     else if  $\varphi = false$  then return false
6:     else if  $\varphi$  is an atom then return the truth value of the atom in  $\pi[0]$ .
7:     else if  $\varphi = \neg\psi$  then return prog( $\pi, \psi$ )
8:     else if  $\varphi = \psi_1 \wedge \psi_2$  then return prog( $\psi_1$ )  $\wedge$  prog( $\psi_2$ )
9:     else if  $\varphi = \psi_1 \vee \psi_2$  then return prog( $\psi_1$ )  $\vee$  prog( $\psi_2$ )
10:    else if  $\varphi = \psi_1 \mathcal{U}_{[a,b]} \psi_2$  then
11:      if  $0 < a \ \& \ a \leq b$  then return  $\psi_1 \mathcal{U}_{[a-1,b-1]} \psi_2$  ▷If the operator is
      not active, return with advanced time.
12:      else if  $0 = a \ \& \ a \leq b$  then return  $\left( \text{prog}(\pi, \psi_2) \vee \left( \text{prog}(\psi_1) \wedge \right. \right.$ 
       $\left. \left. \psi_1 \mathcal{U}_{[0,b-1]} \psi_2 \right) \right)$     ▷If the  $\mathcal{U}$  is active,  $\psi_2$  or  $\psi_1 \mathcal{U}_{[0,b-1]} \psi_2$  could be true.
13:      else if  $0 = a \ \& \ a = b$  then return prog( $\psi_2$ )    ▷When the interval is
      complete, only consider the second subformula.
14:      end if
15:      else if  $\varphi = \psi_1 \mathcal{R}_{[a,b]} \psi_2$  then return  $\neg \text{prog}(\neg \psi_1 \mathcal{U}_{[a,b]} \neg \psi_2)$ 
16:      ▷ Formulas with  $\Box$  and  $\Diamond$  operators are parsed into their logical
      duals when being read into the program and are not implemented.
17:      end if
18:    end if
19: end function

```

Algorithm 2 The Almost-SAT Pattern Algorithm from [1].

```

1: function ALMOST-SAT( $\varphi, K$ )
2:   if SAT( $\varphi$ ) = UNSAT then return
3:   end if
4:   Let  $\pi$  be the model from SAT( $\varphi$ )
5:   while  $1 \leq k \leq |\pi|$  do
6:     if  $|\pi| > K$  then return  $\langle \varphi, \pi, \mathcal{O} \rangle$ 
7:     end if
8:     Let  $\varphi' = \text{prog}(\varphi, \pi_k)$ 
9:     if SAT( $\varphi'$ ) = UNSAT then
10:      Push  $\langle k, \text{false} \rangle$  into  $\mathcal{O}$ 
11:     else
12:      Let  $\pi'$  be the model from SAT( $\varphi'$ )
13:      Update  $\pi = \pi$  appended by  $\pi'$ 
14:      Push  $\langle k, \text{true} \rangle$  into  $\mathcal{O}$ 
15:     end if
16:   end while
17:   return  $\langle \varphi, \pi, \mathcal{O} \rangle$ 
18: end function

```

The Almost-SAT pattern algorithm (Alg. 2) is the primary algorithm that handles repeatedly calling the *prog* algorithm presented above. In the original version of the algorithm, no concern was given to setting upper and lower time bounds to enable usage of the function in a broader context, nor to defining a consistent output format.

References

1. Li, J., Rozier, K.Y.: Mltl benchmark generation via formula progression. In: Colombo, C., Leucker, M. (eds.) Runtime Verification. pp. 426–433. Springer International Publishing, Cham (2018)